



www.mech-lang.org

A Rust-Native Embeddable Reactive Numerical Language for Heterogeneous Computing in Robotics

Corey Montella, Aung Kant Ko, Steven McPhillimey

Motivation

Rust’s memory safety and performance make it a strong foundation for robotics. Still, numerical kernels can require substantial code for SIMD layouts and GPU execution. Backend-specific implementations can be harder to verify against the equations and maintain. Streaming sensor updates also require state management and validation.

This case study presents Mech’s numerical features. The functional language also includes first-class state machines and pattern matching.

Embeddable

Mech runs standalone or embeds in Rust through its native interface. Browser applications can embed it through WebAssembly. Rust hosts can load a dylib or compile Mech source, bind inputs and retain state between updates.

JIT compilation from Rust

```
use mech::kernel::{Backend, Kernel};
let source = include_str!("ekf.mec");
let kernel = Kernel::from_source(source)
    .input("bearing", [-0.55; 4])
    .export("state")
    .compile(Backend::Jit)?;
let mut ekf = kernel.start()?;
ekf.turn([("bearing", [-0.54; 4])]);
let state = ekf.state("state");
```

Compile once, then submit successive measurement updates. Each checked turn retains the last accepted state.

Mech is a Rust-native embeddable language that aims to make numerical kernels easier to write in Rust. For robotics, it adds reactive semantics that process successive sensor updates, validate candidate state and retain the last accepted state when a turn fails. Applications can use one function or a complete subsystem.

Checked dylibs

Artifact	Bytes	M/s ± MAD	Peak RSS (MiB)
Mech scalar	33,544	14.671 ± 0.008	2.69 ± 0.00
Rust scalar	50,016	21.121 ± 0.005	2.69 ± 0.00
Mech SIMD-4	33,864	34.863 ± 0.010	2.69 ± 0.00

Load an AOT library

```
let kernel = unsafe { Kernel::load_bundle("ekf.bundle")? };
let mut ekf = kernel.start()?;
ekf.turn([("bearing", [-0.54; 4])]);
```

The bundle contains the dylib and its saved interface.

Scan the bottom QR code for a fuller demonstration.

Numerical

Mech offers a concrete syntax that expresses numerical algorithms as typed matrix equations. The bearing-only EKF below adapts the textbook formulation [1], preserving its matrix-level notation.

SIMD Rust uses almost five times as many normalized source characters here, mainly for SIMD and batch support. Reusing the Mech source across backends reduces the specialized implementations to maintain.

Mech compiles a typed dataflow artifact. Its ordinary backend uses nalgebra. Other backends specialize the graph without backend-specific source annotations.

Extended Kalman Filter

```
+> math/*
Δt := 0.1
m := [140 12]'
Q := [0.01 0
      0 0.0025]
R := 0.25
I := [1 0 0
      0 1 0
      0 0 1]
~μ := [55 25 0.4]'
~Σ := [100 0 0
      0 100 0
      0 0 0.15]
```

Time Update

```
θ := μ[3]
d := v * Δt
G := [1 0 (0 - d * sin(θ))
      0 1 d * cos(θ)
      0 0 1]
V := [cos(θ) * Δt 0
      sin(θ) * Δt 0
      0 Δt]
```

```
μ̄ := μ + [d * cos(θ)
          d * sin(θ)
          ω * Δt]
```

```
Σ̄ := G ** Σ ** G' + V ** Q ** V'
```

Measurement Update

```
δ := m - μ̄[1..=2]
δx := δ[1]
δy := δ[2]
q := δx ^ 2 + δy ^ 2
```

```
ẑ := atan2(δy, δx) - μ̄[3]
r := z - ẑ
v := atan2(sin(r), cos(r))
```

```
H := [δy / q, (0 - δx) / q, -1]
S := H ** Σ̄ ** H' + R
K := (Σ̄ ** H') / S[1]
A := I - K ** H
```

```
μ₊ := μ̄ + K * v
Σ₊ := A ** Σ̄ ** A' + (K ** K') * R
```

```
μ = μ₊
Σ = Σ₊
```

Reactive

Robots commonly use feedback control loops over continuous data streams. Rust host drivers supply measurements directly or through ROS. Each Mech turn validates candidate state against integrity constraints. Rejection notifies the caller and retains the previous state.

Heterogeneous

Mech’s typed graph supports interpretation, scalar/SIMD CPU, multicore, GPU and WebAssembly/WGPU. Mech backend rows share one f32 EKF implementation of the equations shown at right. Unchecked runs omit integrity predicates and retain per-turn publication. Checking reduces SIMD AOT throughput by 6.6%. The Metal difference is within measured variation.

Mech and Rust show similar throughput

Backend choice changes EKF throughput

Status and Future Work

This poster presents a snapshot of Mech v0.4-beta. Many features are implemented and working, with further validation and hardening needed. Validation on research robots remains limited. Over the next year, we plan broader hardware deployments and user feedback to guide development toward a v1.0 release in 2028.

Experience this poster as a live reactive program. Mech’s WebAssembly runtime supports the CPU and WGPU comparison online.

Benchmark setup. Apple M1: 4P + 4E CPU, 8-core GPU, 8 GB, macOS 15.6.1. Charts: f32, 500,000 filters × 40 turns, n=10. Median ± MAD. Rust 1.96.0-nightly ec818fda3/LLVM 22.1.0. Cranelift 0.131.3. Julia 1.12.6. Mojo 1.1.0/MAX 26.6.0. Futhark 0.27.1/ISPC 1.31.0/LLVM 22.1.8. Python 3.12.14/NumPy 2.2.6/Numba 0.61.2. Clang 17.0.0. Mech v0.4 development (package 0.3.6). Backend pairs f4b69052c. CPU eedc1c75b, dylibs dab98af1d. iros-workshop-benchmarks, benchmarks/iros-2026. † Dylibs: minimal native-ABI hosts, 1 worker, 10,000 filters × 200 turns, n=7. Whole-process peak RSS. Timing excludes compilation, loading and warmup. ‡ SIMD JIT and AOT share the Cranelift pipeline. JIT compiles on demand. AOT emits a library ahead of deployment, allowing the host to omit the compiler. [1] Thrun, Burgard & Fox. Probabilistic Robotics. MIT Press, 2005. §7.4.2, Table 7.2 (adapted).